

Practical-1

Aim:

To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

1) Bubble sort

Input :- Array A with n elements
Output :- Sorted array A in ascending order.
Step 1 :- Start
Step 2 :- Read 'n' and array 'A'
Step 3 :- Set $i = 0$
Step 4 :- Repeat while $i < n - 1$
 Set $j = 0$
 Compare $A[j]$ and $A[j+1]$.
 If $A[j] > A[j+1]$, swap them
 Increment j until $j < n - i - 1$
 Increment i
Step 5 :- Display Sorted Array
Step 6 :- Stop

code:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Flag to check if swapping happens
        bool swapped = false;

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap adjacent elements
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
    }

    // If no swapping occurred, array is already sorted
    if (!swapped)
        break;
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

```
Output
Sorted array: 11 12 22 25 34 64 90
=== Code Execution Successful ===
```

Time complexity:

$$\begin{aligned}
 T(n) &= (n-1) + (n-2) + (n-3) + \dots + 2 + 1 \\
 &= \frac{n(n-1)}{2} \\
 &= \frac{n^2 - n}{2} \\
 T(n) &= n^2 \\
 T(n) &= O(n^2)
 \end{aligned}$$

Best case :-

only 1 pass is made through the array

$$\begin{aligned}
 n-1 \\
 T(n) &= n-1 \\
 T(n) &= n \\
 \boxed{T(n) = O(n)}
 \end{aligned}$$

Avg case :-

$$\begin{aligned}
 \text{comparison} &= \frac{n(n-1)}{2} \\
 \text{Swaps} &= \frac{n(n-1)}{4} \\
 T(n) &= \frac{n(n-1)}{2} + \frac{n(n-1)}{4} \\
 T(n) &= \frac{3}{4}n^2 - \frac{3}{4}n \\
 &= n^2 \\
 T(n) &= O(n^2)
 \end{aligned}$$

worst case :-

$$\begin{aligned}
 \text{comparison} &= (n-1) + (n-2) + \dots + 1 \\
 &= \frac{n(n-1)}{2} \\
 T(n) &= (2+2) + 3 \cdot \left(\frac{n(n-1)}{2} \right) \\
 &= n^2 - n + 2n + 2 \\
 &= n^2 + n + 2 \\
 &= n^2 \\
 T(n) &= O(n^2)
 \end{aligned}$$

2) Selection sort

Input:- Array A with n elements
output:- A sorted in ascending order

Step 1:- for $i \leftarrow 0$ to $n-2$
 $\min \leftarrow i$

Step 2:- for $j \leftarrow i+1$ to $n-1$

Step 3:- if $a[j] < a[\min]$ then
 $\min \leftarrow j$
 END IF
 END for

Step 4:- $\text{Temp} \leftarrow a[i]$
 $a[i] \leftarrow a[\min]$
 $a[\min] \leftarrow \text{Temp}$

Step 5:- END for

Step 6:- Display the sorted Array

Step 7:- stop

code:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find the smallest element
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap smallest element with current element
        swap(arr[i], arr[minIndex]);
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    selectionSort(arr, n);

    cout << "Sorted array(Selection sort): ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

Output

Sorted array(selection sort): 11 12 22 25 64

=== Code Execution Successful ===

Time complexity:

Best case:- Array already sorted
Outer loop $\rightarrow n$ times
Inner loop $\rightarrow T(n) = (n-1) + (n-2) + \dots + 2 + 1$
$$= \frac{n(n-1)}{2}$$
$$= \frac{n^2 - n}{2}$$
$$T(n) = O(n^2)$$

Worst case:- Array sorted in reverse order
Comparisons $= \frac{n(n-1)}{2}$
Swaps $= 0$
$$T(n) = (n-1) + (n-2) + \dots + 2 + 1$$
$$= \frac{n(n-1)}{2}$$
$$= \frac{n^2 - n}{2}$$
$$T(n) = O(n^2)$$

Avg case:-
Comparisons $T(n) = (n-1) + (n-2) + \dots + 2 + 1$
Swaps $= n-1$
$$= \frac{n(n-1)}{2} + n$$
$$= \frac{n^2 - n}{2} + n$$
$$T(n) = n^2$$
$$T(n) = O(n^2)$$

3) Insertion sort

Input :- Array A with n elements.
Output :- Array A sorted in ascending order.
Step 1 :- start.
Step 2 :- Read n and array A.
Step 3 :- $i = 1$
Step 4 :- Set $key = A[i]$ and $j = i - 1$
Step 5 :- while $j \geq 0$ and $A[j] > key$
 shift $A[j]$ to $A[j+1]$
 key at $A[j+1]$
Step 6 :- Display the sorted Array.
Step 7 :- stop.

code:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key at the correct position
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);

    cout << "Sorted array(insertion sort): ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

Output

Sorted array(insertion sort): 5 6 11 12 13

=== Code Execution Successful ===

Time complexity:

Best case:- Array already sorted
only 1 comparison per outer iteration $n-1$
 $T(n) = n-1$
 $T(n) = n$
 $T(n) = O(n)$

worst case:- Array sorted in reverse order
Comparisons $= (1+2+3+ \dots + (n-1))$
 $= \frac{n(n-1)}{2}$
 $T(n) = n + \frac{n(n-1)}{2}$
 $= \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

Average case:-
Comparisons $= \frac{n(n-1)}{4}$
 $T(n) = \frac{n + \frac{n(n-1)}{4}}{4}$
 $= \frac{n + \frac{n^2 - n}{4}}{4}$
 $T(n) = n^2$
 $T(n) = O(n^2)$

4) Merge sort

Input :- Array A with n element

output :- Array A sorted in ascending order

Step 1 :- If $low \leftarrow high$ then
 $mid \leftarrow (low + high) / 2$

Step 2 :- Merge sort (A, low, mid)
Merge sort (A, mid+1, high)
Merge sort (A, low, mid, high)

Step 3 :- END IF
END

Step 4 :- Merge (A, low, mid, high)
Merge the two sorted subarray into a temporary array copy the temporary array back into A.

Step 5 :- END.

code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high) {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = low, k = 0; i <= high; i++, k++) {
        arr[i] = temp[k];
    }
}
```

```
void mergeSort(int arr[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
  
        // Divide  
        mergeSort(arr, low, mid);  
        mergeSort(arr, mid + 1, high);  
  
        // Merge  
        merge(arr, low, mid, high);  
    }  
}
```

```
int main() {  
    int arr[] = {38, 27, 43, 3, 9, 82, 10};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    mergeSort(arr, 0, n - 1);  
  
    cout << "Sorted array(merge sort): ";  
  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

Output

```
Sorted array(merge sort): 3 9 10 27 38 43 82
```

```
=== Code Execution Successful ===
```

Time complexity:

All cases (Best = worst = Average):-

Merge sort always splits the arrays exactly in half and always merges in $O(n)$, regardless of input order.

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

$$= 8T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$n/2^k = 1$$

$$2^k = n$$

$$k = \log_2 n$$

$$k = \log_2 n$$

$$T(n) = nT(1) + n \log_2 n$$

$$= n + n \log n$$

$$T(n) = O(n \log n)$$

5) Quick sort

Input :- Array Element

Output :- Sorted Array

Step 1:- Quick sort (A, low, high)

Step 2:- If low < high then

Pivot $\leftarrow$ partition (A, low, high)

Step 3:- Quick sort (A, low, pivot - 1)

Quick sort (A, pivot + 1, high)

Step 4:- END IF

Step 5:- Partition (A, low, high)

Step 6:- Pivot $\leftarrow$ A[low]

i $\leftarrow$ low
j $\leftarrow$ high

Step 7:- while i < j
while A[i] $\leq$ pivot

i $\leftarrow$ i + 1

END while

Step 8:- If i < j then

Swap A[i] A[j]

Step 9:- END IF

END while

Step 10:- Swap A[low], A[j]

Step 11:- Return J

Step 12:- END

code:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

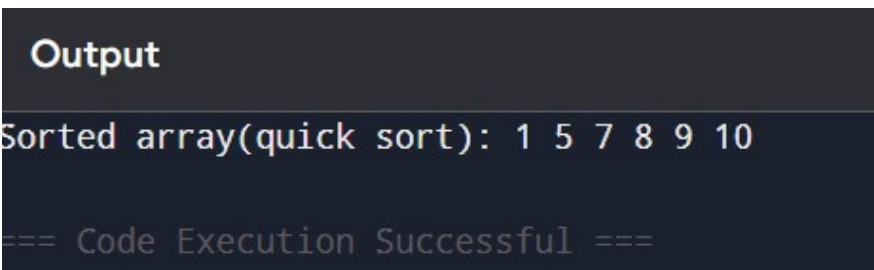
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Divide
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
int main() {  
    int arr[] = {10, 7, 8, 9, 1, 5};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    quickSort(arr, 0, n - 1);  
  
    cout << "Sorted array(Quick sort): ";  
  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:



Output

```
Sorted array(quick sort): 1 5 7 8 9 10
```

=== Code Execution Successful ===

Time complexity:

① Best case:

$$T(n) = n \log_2 n$$

$$T(n) = O(n \log n)$$

② Avg case:

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2[2T(n/4) + n/2] + n$$

$$T(n) = 4T(n/4) + 2n$$

$$T(n) = 8T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$n/2^k = 1$$

$$k = \log_2 n$$

$$T(n) = n \log_2 n$$

$$T(n) = O(n \log n)$$

③ worst case:

Instead of dividing into two equal parts, we

$$T(n) = n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

$$= \frac{n^2 + n}{2}$$

$$T(n) = O(n^2)$$

Conclusion:

We implemented Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort and studied their working and time complexities. Merge Sort and Quick Sort are generally more efficient for large datasets compared to the other three algorithms.
